

where in terms of intermediate variables

$$\begin{aligned}\alpha_1 &= \frac{a_1}{b_1} \\ \alpha_n &= \frac{a_n}{b_n b_{n-1}}, \quad n \geq 2\end{aligned}\tag{5.2.13}$$

we have

$$\begin{aligned}d_0 &= b_0, \quad c_1 = \alpha_1, \quad d_1 = 1 + \alpha_2 \\ c_n &= -\alpha_{2n-1}\alpha_{2n-2}, \quad d_n = 1 + \alpha_{2n-1} + \alpha_{2n}, \quad n \geq 2\end{aligned}\tag{5.2.14}$$

You can find the similar formula for the odd part in the review by Blanch [1]. Often a combination of the transformations (5.2.14) and (5.2.11) is used to get the best form for numerical work.

We will make frequent use of continued fractions in the next chapter.

CITED REFERENCES AND FURTHER READING:

- Abramowitz, M., and Stegun, I.A. 1964, *Handbook of Mathematical Functions* (Washington: National Bureau of Standards); reprinted 1968 (New York: Dover); online at <http://www.nist.gov/aands>, §3.10.
- Blanch, G. 1964, "Numerical Evaluation of Continued Fractions," *SIAM Review*, vol. 6, pp. 383–421.[1]
- Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 11.[2]
- Cuyt, A., and Wuytack, L. 1987, *Nonlinear Methods in Numerical Analysis* (Amsterdam: North-Holland), Chapter 1.
- Fike, C.T. 1968, *Computer Evaluation of Mathematical Functions* (Englewood Cliffs, NJ: Prentice-Hall), §8.2, §10.4, and §10.5.[3]
- Wallis, J. 1695, in *Opera Mathematica*, vol. 1, p. 355, Oxoniae e Theatro Shedoniano. Reprinted by Georg Olms Verlag, Hildesheim, New York (1972).[4]
- Thompson, I.J., and Barnett, A.R. 1986, "Coulomb and Bessel Functions of Complex Arguments and Order," *Journal of Computational Physics*, vol. 64, pp. 490–509.[5]
- Lentz, W.J. 1976, "Generating Bessel Functions in Mie Scattering Calculations Using Continued Fractions," *Applied Optics*, vol. 15, pp. 668–671.[6]
- Jones, W.B. 1973, in *Padé Approximants and Their Applications*, P.R. Graves-Morris, ed. (London: Academic Press), p. 125.[7]

5.3 Series and Their Convergence

Everybody knows that an analytic function can be expanded in the neighborhood of a point x_0 in a power series,

$$f(x) = \sum_{k=0}^{\infty} a_k (x - x_0)^k\tag{5.3.1}$$

Such series are straightforward to evaluate. You don't, of course, evaluate the k th power of $x - x_0$ *ab initio* for each term; rather, you keep the $k - 1$ st power and update

it with a multiply. Similarly, the form of the coefficients a_k is often such as to make use of previous work: Terms like $k!$ or $(2k)!$ can be updated in a multiply or two.

How do you know when you have summed enough terms? In practice, the terms had better be getting small fast, otherwise the series is not a good technique to use in the first place. While not mathematically rigorous in all cases, standard practice is to quit when the term you have just added is smaller in magnitude than some small ϵ times the magnitude of the sum thus far accumulated. (But watch out if isolated instances of $a_k = 0$ are possible!)

Sometimes you will want to compute a function from a series representation even when the computation is *not* efficient. For example, you may be using the values obtained to fit the function to an approximating form that you will use subsequently (cf. §5.8). If you are summing very large numbers of slowly convergent terms, pay attention to roundoff errors! In floating-point representation it is more accurate to sum a list of numbers in the order starting with the smallest one, rather than starting with the largest one. It is even better to group terms pairwise, then in pairs of pairs, etc., so that all additions involve operands of comparable magnitude.

A weakness of a power series representation is that it is guaranteed *not* to converge farther than that distance from x_0 at which a singularity is encountered *in the complex plane*. This catastrophe is not usually unexpected: When you find a power series in a book (or when you work one out yourself), you will generally also know the radius of convergence. An insidious problem occurs with series that converge everywhere (in the mathematical sense), but almost nowhere fast enough to be useful in a numerical method. Two familiar examples are the sine function and the Bessel function of the first kind,

$$\sin x = \sum_{k=0}^{\infty} \frac{(-1)^k}{(2k+1)!} x^{2k+1} \quad (5.3.2)$$

$$J_n(x) = \left(\frac{x}{2}\right)^n \sum_{k=0}^{\infty} \frac{(-\frac{1}{4}x^2)^k}{k!(k+n)!} \quad (5.3.3)$$

Both of these series converge for all x . But both don't even start to converge until $k \gg |x|$; before this, their terms are increasing. Even worse, the terms alternate in sign, leading to large cancellation errors with finite precision arithmetic. This makes these series useless for large x .

5.3.1 Divergent Series

Divergent series are often very useful. One class consists of power series outside their radius of convergence, which can often be summed by the acceleration techniques we will describe below. Another class is asymptotic series, such as the Euler series that comes from Euler's integral (related to the exponential integral E_1):

$$E(x) = \int_0^{\infty} \frac{e^{-t}}{1+xt} dt \simeq \sum_{k=0}^{\infty} (-1)^k k! x^k \quad (5.3.4)$$

Here the series is derived by expanding $(1+xt)^{-1}$ in powers of x and integrating term by term. The series diverges for all $x \neq 0$. For $x = 0.1$, the series gives only three significant digits before diverging. Nevertheless, convergence acceleration

techniques allow effortless evaluation of the function $E(x)$, even for $x \sim 2$, when the series is wildly divergent!

5.3.2 Accelerating the Convergence of Series

There are several tricks for accelerating the rate of convergence of a series or, equivalently, of a sequence of partial sums

$$s_n = \sum_{k=0}^n a_k \quad (5.3.5)$$

(We'll use the terms sequence and series interchangeably in this section.) An excellent review has been given by Weniger [1]. Before we can describe the tricks and when to use them, we need to classify some of the ways in which a sequence can converge. Suppose s_n converges to s , say, and that

$$\lim_{n \rightarrow \infty} \frac{a_{n+1}}{a_n} = \rho \quad (5.3.6)$$

If $0 < |\rho| < 1$, we say the convergence is *linear*; if $\rho = 1$, it is *logarithmic*; and if $\rho = 0$, it is *hyperlinear*. Of course, if $|\rho| > 1$, the sequence diverges. (More rigorously, this classification should be given in terms of the so-called remainders $s_n - s$ [1]. However, our definition is more practical and is equivalent if we restrict the logarithmic case to terms of the same sign.)

The prototype of linear convergence is a geometric series,

$$s_n = \sum_{k=0}^n x^k = \frac{1 - x^{n+1}}{1 - x} \quad (5.3.7)$$

It is easy to see that $\rho = x$, and so we have linear convergence for $0 < |x| < 1$. The prototype of logarithmic convergence is the series for the Riemann zeta function,

$$\zeta(x) = \sum_{k=1}^{\infty} \frac{1}{k^x}, \quad x > 1 \quad (5.3.8)$$

which is notoriously slowly convergent, especially as $x \rightarrow 1$. The series (5.3.2) and (5.3.3), or the series for e^x , exemplify hyperlinear convergence. We see that hyperlinear convergence doesn't necessarily imply that the series is easy to evaluate for all values of x . Sometimes convergence acceleration is helpful only once the terms start decreasing.

Probably the most famous series transformation for accelerating convergence is the Euler transformation (see, e.g., [2,3]), which dates from 1755. Euler's transformation works on *alternating series* (where the terms in the sum alternate in sign). Generally it is advisable to do a small number of terms directly, through term $n - 1$, say, and then apply the transformation to the rest of the series beginning with term n . The formula (for n even) is

$$\sum_{s=0}^{\infty} (-1)^s a_s = a_0 - a_1 + a_2 - \dots - a_{n-1} + \sum_{s=0}^{\infty} \frac{(-1)^s}{2^{s+1}} [\Delta^s a_n] \quad (5.3.9)$$

Here Δ is the *forward difference operator*, i.e.,

$$\begin{aligned}\Delta a_n &\equiv a_{n+1} - a_n \\ \Delta^2 a_n &\equiv a_{n+2} - 2a_{n+1} + a_n \\ \Delta^3 a_n &\equiv a_{n+3} - 3a_{n+2} + 3a_{n+1} - a_n \quad \text{etc.}\end{aligned}\tag{5.3.10}$$

Of course you don't actually do the infinite sum on the right-hand side of (5.3.9), but only the first, say, p terms, thus requiring the first p differences (5.3.10) obtained from the terms starting at a_n . There is an elegant and subtle implementation of Euler's transformation due to van Wijngaarden [6], discussed in full in a Webnote [7].

Euler's transformation is an example of a *linear* transformation: The partial sums of the transformed series are linear combinations of the partial sums of the original series. Euler's transformation and other linear transformations, while still important theoretically, have generally been superseded by newer *nonlinear* transformations that are considerably more powerful. As usual in numerical work, there is no free lunch: While the nonlinear transformations are more powerful, they are somewhat riskier than linear transformations in that they can occasionally fail spectacularly. But if you follow the guidance below, we think that you will never again resort to puny linear transformations.

The oldest example of a nonlinear sequence transformation is *Aitken's Δ^2 -process*. If s_n, s_{n+1}, s_{n+2} are three successive partial sums, then an improved estimate is

$$s'_n \equiv s_n - \frac{(s_{n+1} - s_n)^2}{s_{n+2} - 2s_{n+1} + s_n} = s_n - \frac{(\Delta s_n)^2}{\Delta^2 s_n}\tag{5.3.11}$$

The formula (5.3.11) is exact for a geometric series, which is one way of deriving it. If you form the sequence of s'_i 's, you can apply (5.3.11) a second time to *that* sequence, and so on. (In practice, this iteration will only rarely do much for you after the first stage.) Note that equation (5.3.11) should be computed as written; there exist algebraically equivalent forms that are much more susceptible to roundoff error.

Aitken's Δ^2 -process works only on linearly convergent sequences. Like Euler's transformation, it has also been superseded by algorithms such as the two we will now describe. After giving routines for these algorithms, we will supply some rules of thumb on when to use them.

The first "modern" nonlinear transformation was proposed by Shanks. An efficient recursive implementation was given by Wynn, called the ϵ algorithm. Aitken's Δ^2 -process is a special case of the ϵ algorithm, corresponding to using just three terms at a time. Although we will not give a derivation here, it is easy to state exactly what the ϵ algorithm does: If you input the partial sums of a power series, the ϵ algorithm returns the "diagonal" Padé approximants (§5.12) evaluated at the value of x used in the power series. (The coefficients in the approximant itself are not calculated.) That is, if $[M/N]$ denotes the Padé approximant with a polynomial of degree M in the numerator and degree N in the denominator, the algorithm returns the numerical values of the approximants

$$[0, 0], \quad [1/0], \quad [1/1], \quad [2/1], \quad [2/2], \quad [3, 2], \quad [3, 3] \quad \dots\tag{5.3.12}$$

(The object `Epsalg` below is roughly equivalent to `pade` in §5.12 followed by an evaluation of the resulting rational function.)

In the object `Epsalg`, which is based on a routine in [1], you supply the sequence term by term and monitor the output for convergence in the calling program. Internally, the routine contains a check for division by zero and substitutes a large number

for the result. There are three conditions under which this check can be triggered: (i) Most likely, the algorithm has already converged, and should have been stopped earlier; (ii) there is an “accidental” zero term, and the program will recover; (iii) hardly ever in practice, the algorithm can actually fail because of a perverse combination of terms. Because (i) and (ii) are vastly more common than (iii), `Epsalg` hides the check condition and instead returns the last-known good estimate.

`struct Epsalg {`

Convergence acceleration of a sequence by the ϵ algorithm. Initialize by calling the constructor with arguments `nmax`, an upper bound on the number of terms to be summed, and `epss`, the desired accuracy. Then make successive calls to the function `next`, with argument the next partial sum of the sequence. The current estimate of the limit of the sequence is returned by `next`. The flag `cnvgd` is set when convergence is detected.

`series.h`

```

    VecDoub e;                               Workspace.
    Int n,ncv;
    Bool cnvgd;
    Doub eps,small,big,lastval,laststeps;      Numbers near machine underflow and
                                                overflow limits.

    Epsalg(Int nmax, Doub epss) : e(nmax), n(0), ncv(0),
    cnvgd(0), eps(epss), lastval(0.) {
        small = numeric_limits<Doub>::min()*10.0;
        big = numeric_limits<Doub>::max();
    }

    Doub next(Doub sum) {
        Doub diff,temp1,temp2,val;
        e[n]=sum;
        temp2=0.0;
        for (Int j=n; j>0; j--) {
            temp1=temp2;
            temp2=e[j-1];
            diff=e[j]-temp2;
            if (abs(diff) <= small)
                e[j-1]=big;
            else
                e[j-1]=temp1+1.0/diff;
        }
        n++;
        val = (n & 1) ? e[0] : e[1];
        if (abs(val) > 0.01*big) val = lastval;
        laststeps = abs(val-lastval);
        if (laststeps > eps) ncv = 0;
        else ncv++;
        if (ncv >= 3) cnvgd = 1;
        return (lastval = val);
    }
};

```

The last few lines above implement a simple criterion for deciding whether the sequence has converged. For problems whose convergence is robust, you can simply put your calls to `next` inside a `while` loop like this:

```

Doub val, partialsum, eps=...;
Epsalg mysum(1000,eps);
while (! mysum.cnvgd) {
    partialsum = ...
    val = mysum.next(partialsum);
}

```

For more delicate cases, you can ignore the `cnvkd` flag and just keep calling `next` until you are satisfied with the convergence.

A large class of modern nonlinear transformations can be derived by using the concept of a *model sequence*. The idea is to choose a “simple” sequence that approximates the asymptotic form of the given sequence and construct a transformation that sums the model sequence exactly. Presumably the transformation will work well for other sequences with similar asymptotic properties. For example, a geometric series provides the model sequence for Aitken’s Δ^2 -process.

The *Levin transformation* is probably the best single sequence acceleration method currently known. It is based on approximating a sequence asymptotically by an expression of the form

$$s_n = s + \omega_n \sum_{j=0}^{k-1} \frac{c_j}{(n + \beta)^j} \quad (5.3.13)$$

Here ω_n is the dominant term in the remainder of the sequence:

$$s_n - s = \omega_n [c + O(n^{-1})], \quad n \rightarrow \infty \quad (5.3.14)$$

The constants c_j are arbitrary, and β is a parameter that is restricted to be positive. Levin showed that for a model sequence of the form (5.3.13), the following transformation gives the exact value of the series:

$$s = \frac{\sum_{j=0}^k (-1)^j \binom{k}{j} \frac{(\beta + n + j)^{k-1}}{(\beta + n + k)^{k-1}} \frac{s_{n+j}}{\omega_{n+j}}}{\sum_{j=0}^k (-1)^j \binom{k}{j} \frac{(\beta + n + j)^{k-1}}{(\beta + n + k)^{k-1}} \frac{1}{\omega_{n+j}}} \quad (5.3.15)$$

(The common factor $(\beta + n + k)^{k-1}$ in the numerator and denominator reduces the chances of overflow for large k .) A derivation of equation (5.3.15) is given in a Webnote [4].

The numerator and denominator in (5.3.15) are not computed as written. Instead, they can be computed efficiently from a single recurrence relation with different starting values (see [1] for a derivation):

$$D_{k+1}^n(\beta) = D_k^{n+1}(\beta) - \frac{(\beta + n)(\beta + n + k)^{k-1}}{(\beta + n + k + 1)^k} D_k^n(\beta) \quad (5.3.16)$$

The starting values are

$$D_0^n(\beta) = \begin{cases} s_n/\omega_n, & \text{numerator} \\ 1/\omega_n, & \text{denominator} \end{cases} \quad (5.3.17)$$

Although D_k^n is a two-dimensional object, the recurrence can be coded in a one-dimensional array proceeding up the counterdiagonal $n + k = \text{constant}$.

The choice (5.3.14) doesn’t determine ω_n uniquely, but if you have analytic information about your series, this is where you can make use of it. Usually you won’t

be so lucky, in which case you can make a choice based on heuristics. For example, the remainder in an alternating series is approximately half the first neglected term, which suggests setting ω_n equal to a_n or a_{n+1} . These are called the Levin t and d transformations, respectively. Similarly, the remainder for a geometric series is the difference between the partial sum (5.3.7) and its limit $1/(1-x)$. This can be written as $a_n a_{n+1}/(a_n - a_{n+1})$, which defines the Levin v transformation. The most popular choice comes from approximating the remainder in the ζ function (5.3.8) by an integral:

$$\sum_{k=n+1}^{\infty} \frac{1}{k^x} \approx \int_{n+1}^{\infty} \frac{dk}{k^x} = \frac{(n+1)^{1-x}}{x-1} = \frac{(n+1)a_{n+1}}{x-1} \quad (5.3.18)$$

This motivates the choice $(n+\beta)a_n$ (Levin u transformation), where β is usually chosen to be 1. To summarize:

$$\omega_n = \begin{cases} (\beta+n)a_n, & u \text{ transformation} \\ a_n, & t \text{ transformation} \\ a_{n+1}, & d \text{ transformation (modified } t \text{ transformation)} \\ \frac{a_n a_{n+1}}{a_n - a_{n+1}}, & v \text{ transformation} \end{cases} \quad (5.3.19)$$

For sequences that are not partial sums, so that the individual a_n 's are not defined, replace a_n by Δs_{n-1} in (5.3.19).

Here is the routine for Levin's transformation, also based on the routine in [1]:

`struct Levin {`

Convergence acceleration of a sequence by the Levin transformation. Initialize by calling the constructor with arguments `nmax`, an upper bound on the number of terms to be summed, and `epss`, the desired accuracy. Then make successive calls to the function `next`, which returns the current estimate of the limit of the sequence. The flag `cnvgd` is set when convergence is detected.

```
VecDoub numer,denom;      Numerator and denominator computed via (5.3.16).
Int n,ncv;
Bool cnvgd;
Doub small,big;           Numbers near machine underflow and overflow limits.
Doub eps,lastval,laststep;
```

```
Levin(Int nmax, Doub epss) : numer(nmax), denom(nmax), n(0), ncv(0),
cnvgd(0), eps(epss), lastval(0.) {
    small=numeric_limits<Doub>::min()*10.0;
    big=numeric_limits<Doub>::max();
}
```

```
Doub next(Doub sum, Doub omega, Doub beta=1.) {
```

Arguments: `sum`, the n th partial sum of the sequence; `omega`, the n th remainder estimate ω_n , usually from (5.3.19); and the parameter `beta`, which should usually be set to 1, but sometimes 0.5 works better. The current estimate of the limit of the sequence is returned.

```
Int j;
Doub fact,ratio,term,val;
term=1.0/(beta+n);
denom[n]=term/omega;
numer[n]=sum*denom[n];
if (n > 0) {
    ratio=(beta+n-1)*term;
    for (j=1;j<=n;j++) {
```

[series.h](#)